CEG5101 实验作业报告

CA2 异常感知路由与源地址封锁控制器
2026—2027 学年第一学期 | 实验日期：2026 年 10 月 11、16、17 日

摘要

本报告围绕教师提供的六交换机网状拓扑，实现并评价集成 OpenFlow 控制器。控制器监测源流量，以指数移动平均建立基线并识别异常，封锁异常 IPv4 源地址，更新有向链路代价，并在替代路线更便宜时切换合法流量。正文按照实验作业 Task 1 至 Task 5 组织 [1]。

共分析五轮学生亲自操作的实验：A、C 场景各一轮，B 场景三轮。在 h1→h6 提供 10 Mbps UDP 流量的条件下，B 场景由 s1→s2→s6 切换至 s1→s2→s3→s6，并在异常结束后恢复。A、C 的异常链路不在合法原路上，因此合法路线保持不变。所有一秒接收窗口均有流量。B 在异常启动附近短暂丢包，三轮整轮丢包率为 0.0843%、0.1529%、0.0431%；A、C 未观察到 UDP 丢包。

C 场景将 h5→h6 测试对显式固定经过直接 s5→s6 链路。第 4、5 节说明这一实验条件；该测试对未使用无约束最短路线。结果仅支持当前测试负载下的行为，不声称参数最优，也不声称已证明改道相对于仅封锁的独立性能收益。

报告结构

1 拓扑与正常基线 2 监测与封锁 3 动态代价 4 改道机制与三场景 5 性能评价 6 结论与声明 附录：复现与证据索引

1 Task 1 在 MiniEdit 中搭建拓扑

教师提供的 ceg_mesh_topo.py 定义了网络 [2]。按照该文件在 MiniEdit 中重建主机、交换机和链路，保存为 ca2-mesh-my.mn，重新打开并运行。网络包含六台主机、六台交换机和一个远程控制器。17 条数据链路由六条主机接入链路和十一条交换机链路组成；图中的红色控制连接不计入数据链路。

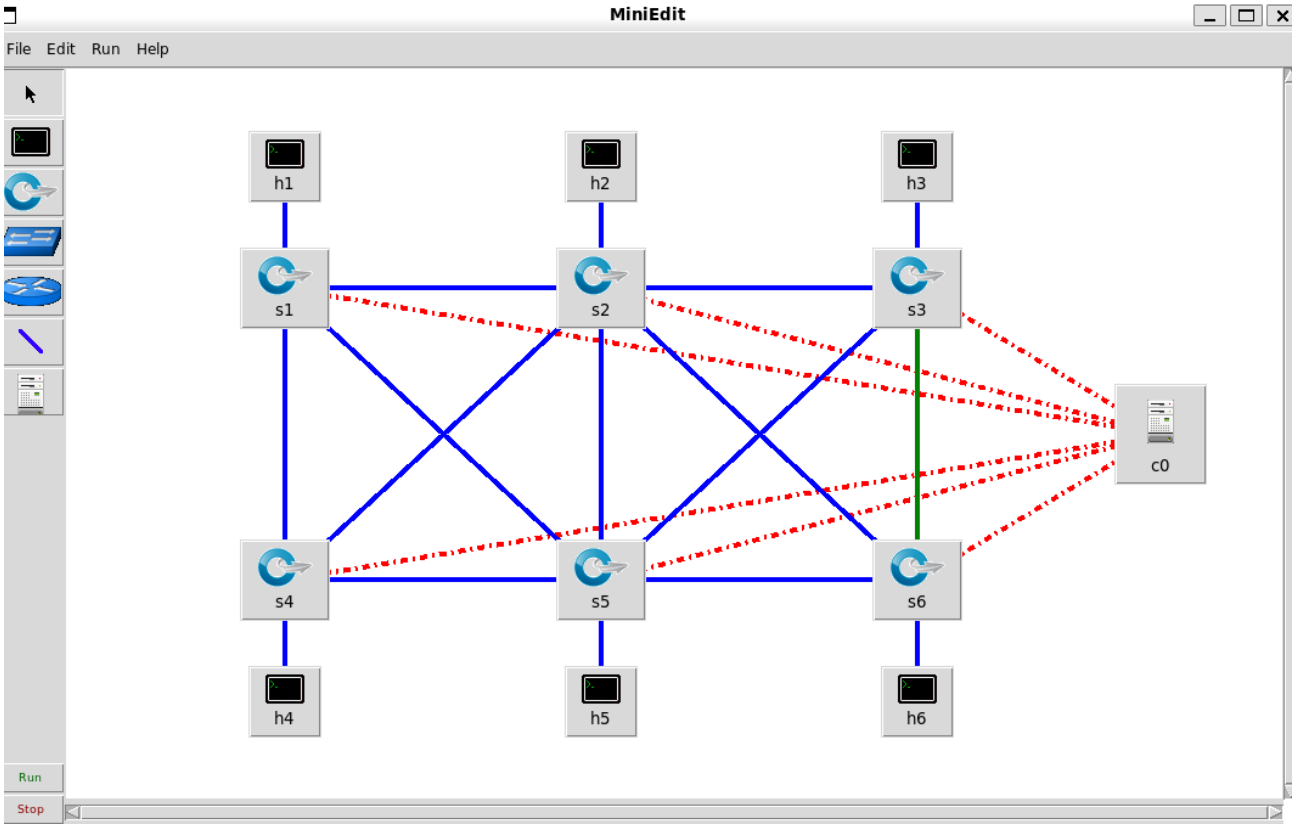


图 1 学生保存的 MiniEdit 拓扑原图，包含 h1 至 h6、s1 至 s6 和 c0。

配置项	取值
主机地址	hi = 10.0.0.i/24, i = 1…6
主机 MAC	00:00:00:00:00:01 … 00:00:00:00:00:06
交换机标识	s1…s6 对应 DPID 1…6，编码为 16 位十六进制数
控制协议	OpenFlow 1.3；c0 → 127.0.0.1:6653
带宽与接入延迟	全部 17 条链路为 40 Mbps；每条主机链路延迟 15 ms

hi 接入 si 的端口 1。MiniEdit 的链路属性设置 TCLink 带宽及延迟，交换机属性设置 DPID 和 OpenFlow 版本。实验同时核对保存文件、Linux 实际接口和 Open vSwitch 状态，因此配置依据不仅是拓扑示意图。

1 链路参数与正常基线

链路	延迟 ms 与基础代价	链路	延迟 ms 与基础代价
s1 – s2	5	s2 – s3	10
s4 – s5	15	s5 – s6	20
s1 – s4	8	s2 – s5	12
s3 – s6	7	s1 – s5	18
s2 – s4	6	s2 – s6	14
s3 – s5	9	—	—

每条交换机链路使用表中的模拟延迟与原始路由权重。每条物理链路建模为两个有向图边，共 22 个有向交换机边。h1→h6 原路为 s1→s2→s6，基础代价 5 + 14 = 19；替代路线 s1→s2→s3→s6 的基础代价为 5 + 10 + 7 = 22。这里计算的是选路权重，实测往返时延还包含两端接入链路、返回路线和调度影响。

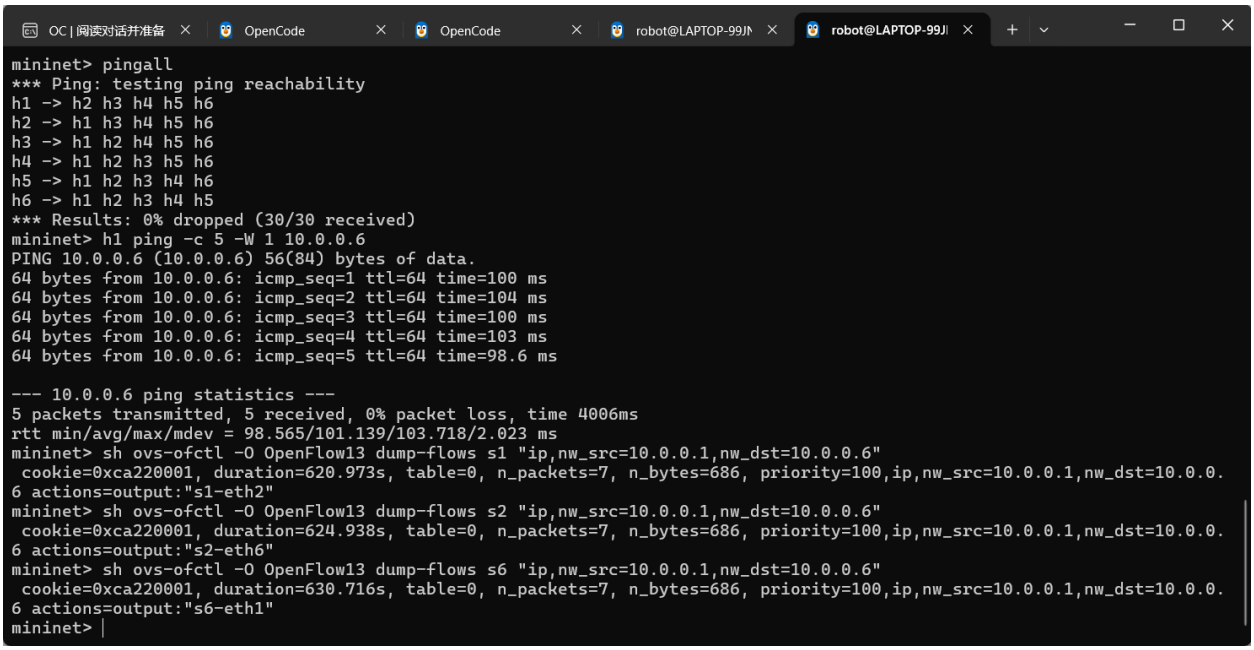


图 2 正常基线流表证据。s1-eth2 通往 s2，s2-eth6 通往 s6，s6-eth1 通往 h6。

基线控制器通过 LLDP 发现交换机及链路，使用固定主机接入信息和代理 ARP 避免网状广播环路，并为 30 个不同源目的有向主机对安装 Dijkstra 路线。pingall 收到 30/30；h1→h6 收到 5/5，平均 RTT 为 101.139 ms。两轮独立的 15 秒、10 Mbps UDP 基线均报告 10.0 Mbps，丢包均为 0/12,759。该基线测试发生在集成异常实验之前 [5]。

2 Task 2 流量监测 异常检测与封锁

Task 2 每秒轮询流表统计；端口统计在 Task 3 加入。每个源的包速率只在源主机接入交换机统计，避免同一数据包沿途经过多台交换机时被重复累计。速率由累计计数按实际时间差分得到；计数器重置时重新建立差分起点，分段统计回复先汇总再处理。

$$r_s(k) = \Delta N_s / \Delta t$$
$$T_s(k) = \max(2000,\ 3E_s(k-1))$$

$$E_s(k) = 0.3r_s(k) + 0.7E_s(k-1)$$

至少五个有效预热样本后，若 $r_s > T_s$，则标记异常源。判断使用更新前的 EMA，异常或已封锁样本不更新 EMA。2000 pps 下限避免空闲或低速基线使普通 10 Mbps UDP 被误判。检测排除 ICMP echo reply，避免将响应攻击请求的受害主机误判为攻击源。这些设置是在教师 EMA 示例 [3] 基础上的实现选择。

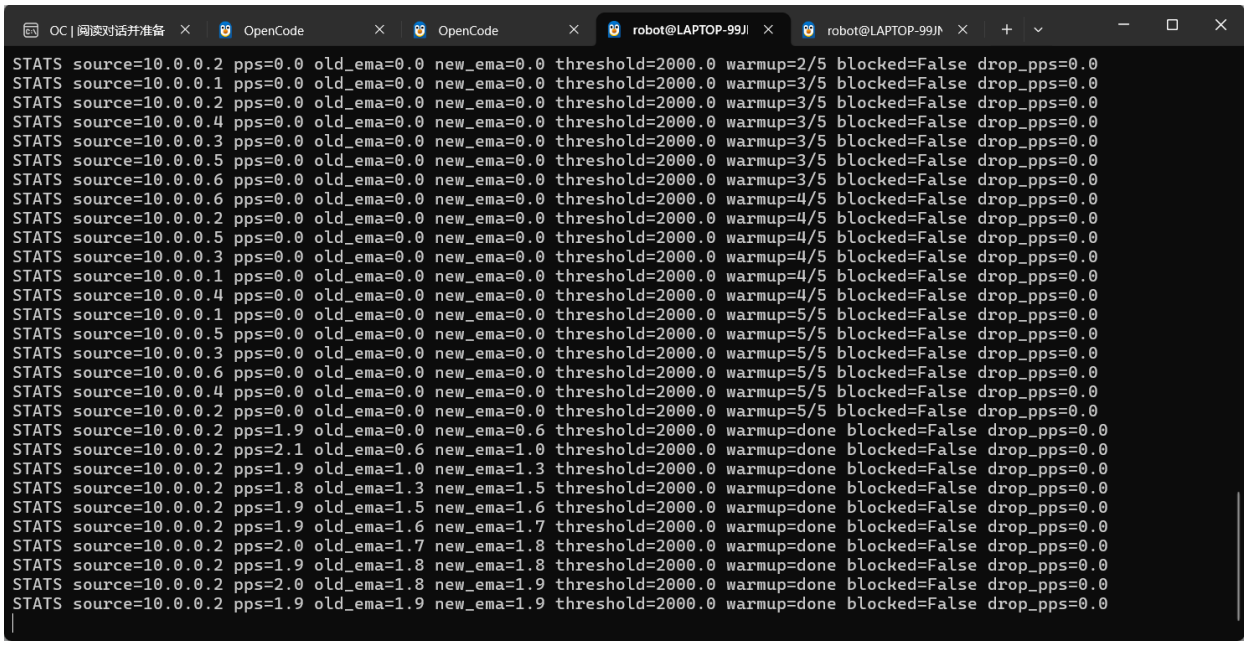


图 3 h2 正常 ping 约为 2 pps，EMA 逐步接近 1.9 pps，未触发封锁。

8 秒 h2→h6 ICMP flood 的检测速率为 13,339.8 pps，高于 2000 pps 阈值。控制器在全部六台交换机安装 priority 5000 的 IPv4 源地址丢弃规则。检查时 s2 已丢弃 478,615 包；之后 h2 ping 收到 0/3，而 h1 收到 3/3。完整后台观测中合法 ping 为 292/293，因此本阶段不能描述为零丢包。高优先级源封锁规则覆盖普通转发规则 [6]。

2 Task 2 从发包命令到交换机封锁规则

终端 A 的 run_anomaly.sh 启动 Ryu 并加载 anomaly_controller.py；终端 B 运行 MiniEdit/Mininet 并产生测试流量。启动控制器本身不制造异常。学生在 Mininet CLI 中使用的原命令如下：

```
h2 timeout 8s hping3 -1 --flood 10.0.0.6 > /tmp/ceg5101-todo03-h2-flood-20261016.log 2>&1
```

部分	含义
h2	在 h2 的网络环境中执行，使用 h2 的接口和源地址。
timeout 8s	8 秒后终止本轮发包程序。
hping3 -1 --flood	高速发送 ICMP 请求；-1 与 --icmp 等价。
10.0.0.6	目的主机 h6。
> /tmp/...log	将标准输出写入日志；同名文件会被覆盖。
2>&1	把标准错误（2）也送到标准输出（1）当前指向的日志。

控制器通过什么连接下规则

各 Open vSwitch 根据 c0 配置连接 Ryu 的 TCP 地址 127.0.0.1:6653，使用 OpenFlow 1.3。Ryu 的 Datapath 对象（dp）关联该交换机的控制连接。FlowStats 请求/回复传递计数；普通 h2→h6 包由交换机转发。traffic_stats.py 的 SourceEMA.observe() 判断异常后，anomaly_controller.py 的 flow_stats() 对每台交换机调用 drop(dp, ip)。

drop() 构造 OFPFlowMod：优先级 5000，匹配 eth_type=0x0800（IPv4）和 ipv4_src=10.0.0.2，动作列表为空且无下一表指令。dp.send_msg() 序列化消息，Ryu 经 TCP 控制连接发送；交换机接收并安装 drop 条目。Python 定义规则，交换机在自己的流表中执行。

为什么 h2→h6 被阻断，h1→h6 仍通

```
Before: h2-eth0 -> s2-eth1 [s2] s2-eth6 -> s6 -> h6
After: h2-eth0 -> s2-eth1 [s2: match source 10.0.0.2, DROP]
```

h2 请求的源 IP 为 10.0.0.2、目的为 10.0.0.6。在 s2，它同时满足原 priority 110 的 ICMP 转发规则（从 s2-eth6 发出）和新 priority 5000 的源地址 drop。5000 优先，s2 丢弃该包，h6 收不到请求，因而没有对应回复。h1 的源 IP 是 10.0.0.1，不匹配 drop，继续正常转发。封锁针对 h2 发出的 IPv4 包，目的不限；s2 – s6 链路及 h6 并未关闭。

实测证据：s2 drop 计数 478,615；随后 h2 ping 为 0/3、h1 为 3/3。检测及安装前已有部分包通过，旧计数仍保留。仅有 BLOCK 日志或 8 秒后发包退出不足以证明封锁。Task 2 保留封锁直到重启本轮。

3 Task 3 动态链路代价更新

每条有向交换机链路独立计算代价。R_{ij} 由目的交换机对应端口 rx_bytes 的增量获得，表示沿该有向链路实际到达的流量；源端口 tx_bytes 作为辅助诊断记录。按照作业中的“负载项加异常项”形式 [1]，本实现使用以下参数：EMA 系数记为 0，负载权重取 5。

$$U_{ij}(t) = \mathrm{clip}(R_{ij}(t) / 40{,}000{,}000,\ 0,\ 1)$$

$$C_{ij}(t) = C^{0}_{ij} + 5U_{ij}(t) + 60A_{ij}(t)$$

检测到的攻击路径上，惩罚生效期间的有向边取 $A_{ij} = 1$，否则取 0。惩罚至少保持 8 秒，并在源入口 drop 计数连续 5 秒无活动后才撤销。统计过期不能作为撤销惩罚的依据。这样可避免“攻击被入口封锁、远端链路不再收到攻击包”导致代价立刻恢复。链路代价恢复后，源地址封锁仍保留。

s2→s6 状态	观测代价
空闲	14.000
合法 10 Mbps UDP	≈15.24 – 15.34
异常惩罚生效	≈75 – 76.637
撤销惩罚 仍有合法负载	15.285878
数据流量结束	≈14.000

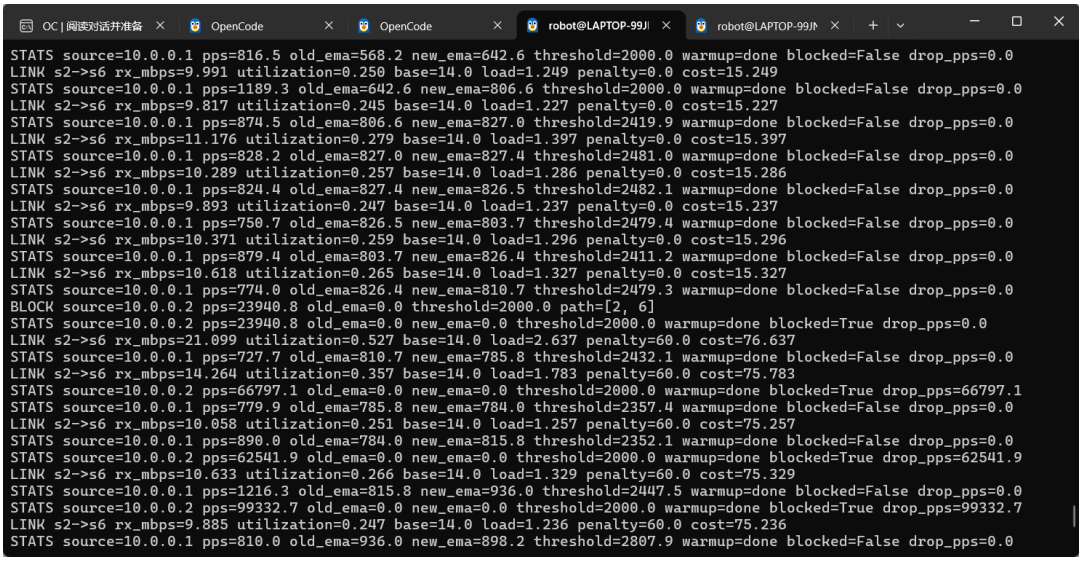


图 4 动态代价阶段：异常源被封锁，s2→s6 增加 60 单位惩罚。

这一独立代价测试有意保留静态转发。60 秒 UDP 报告为 9.99 Mbps，丢失 44/51,024 个数据报，约 0.086%。它验证代价变化，不用于证明改道；Task 4 的集成控制器随后启用路线更新 [7]。

4 Task 4 集成改道设计与实验流程

选择并安装路线

控制器在教师路由示例 [4] 基础上，以当前有向代价运行 Dijkstra。对未被封锁的源目的对，只有新路线至少便宜 1 单位、且距离上次切换至少 3 秒时才更新。改善门槛和保持时间减少轻微负载波动引起的反复切换。源封锁优先级为 5000，高于入口转发的 100、109、110 和带标签中转规则的 200。

路线更新使用内部 VLAN 版本。先准备新标签对应的下游规则，并等待 OpenFlow barrier 确认；再修改源入口的无标签规则，为新包附加新标签并选择新出口，随后再次确认 barrier。目的交换机去掉标签。旧版本保留 5 秒后按专用 cookie 删除。这一顺序使在途包可继续使用旧路线，并保留源统计与封锁规则。有限保留时间是工程保护措施，不构成无条件零丢包保证。

统一实验时间线

时间段	操作
0 – 12 s	持续运行 h1→h6 的 10 Mbps UDP，并同时 ping
12 – 26 s	按场景运行持续 14 秒的 hping3 ICMP flood
26 – 60 s	异常发送停止，观察代价及合法路线恢复
60 秒之后	测试合法连通性与异常源仍被封锁

A 为 h2→h3，对应 s2→s3；B 为 h2→h6，对应 s2→s6；C 为 h5→h6，对应 s5→s6。源封锁持续保留，因此每轮使用重新启动的控制器和拓扑状态。iperf2 发送 1470 字节 UDP 数据报，接收端按秒报告。每秒核对实际流表动作、VLAN 标签、出口端口连接与包计数，确认 h1→h6 已安装的真实路线。

C 场景需要显式施压条件：仅将 h5→h6 IPv4 测试对固定经过直接 s5→s6。无此限制时，其原始最短路为 s5→s3→s6，代价 9 + 7 = 16，低于直接链路的 20。其他主机对仍动态选路，原始链路权重不变。这一测试约束仍需教师或助教确认。

4 场景 A 异常位于 s2 至 s3

由 h2→h3 产生 flood，目标链路为 s2→s3。检测触发源封锁，并对目标边施加异常惩罚。目标边空闲、惩罚生效和恢复后的代价依次为 10 → 约 70 → 10。正常合法流量经过的链路还包含实测负载项。

目标链路采样到达速率峰值为 8.68 Mbps；异常源检测速率为 16,455.6 pps，阈值为 2000 pps。入口封锁且在途流量排空后，目标链路到达速率降至接近空闲，而惩罚继续保持至满足安静期。

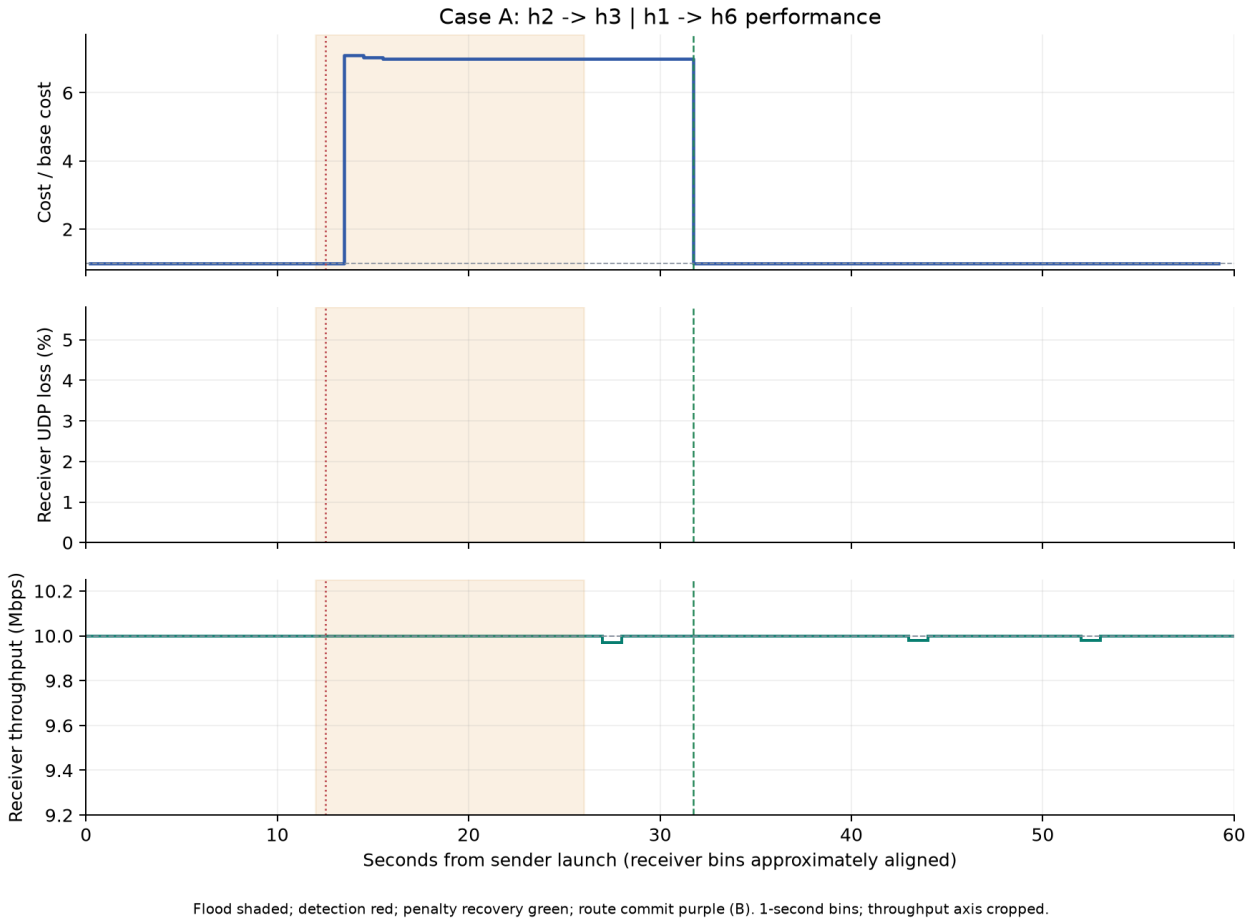


图 5 场景 A 的目标链路归一化代价、接收端 UDP 丢包率和吞吐。B 图使用第 1 轮。阴影表示计划 12–26 秒 flood 时段，事件标记来自控制器记录。

合法路线保持 s1→s2→s6。原路不经过 s2→s3，未受影响的原路仍然更优。因此不能因为网络其他位置发生异常就要求合法流量一定改道。

60 秒接收报告为 10 Mbps，UDP 丢失 0/51,024（0.0000%）。恢复核验时源入口 drop 计数为 905,435。测量后合法 ping 收到 3/3，异常源收到 0/2；所有采样交换机的源封锁规则均保留。

4 场景 B 异常位于 s2 至 s6

由 h2→h6 产生 flood，目标链路为 s2→s6。检测触发源封锁，并对目标边施加异常惩罚。目标边空闲、惩罚生效和恢复后的代价依次为 14 → 约 75–77 → 14。正常合法流量经过的链路还包含实测负载项。

目标链路采样到达速率峰值为 25.01 Mbps；异常源检测速率为 36,017.9 pps，阈值为 2000 pps。入口封锁且在途流量排空后，目标链路到达速率降至接近空闲，而惩罚继续保持至满足安静期。

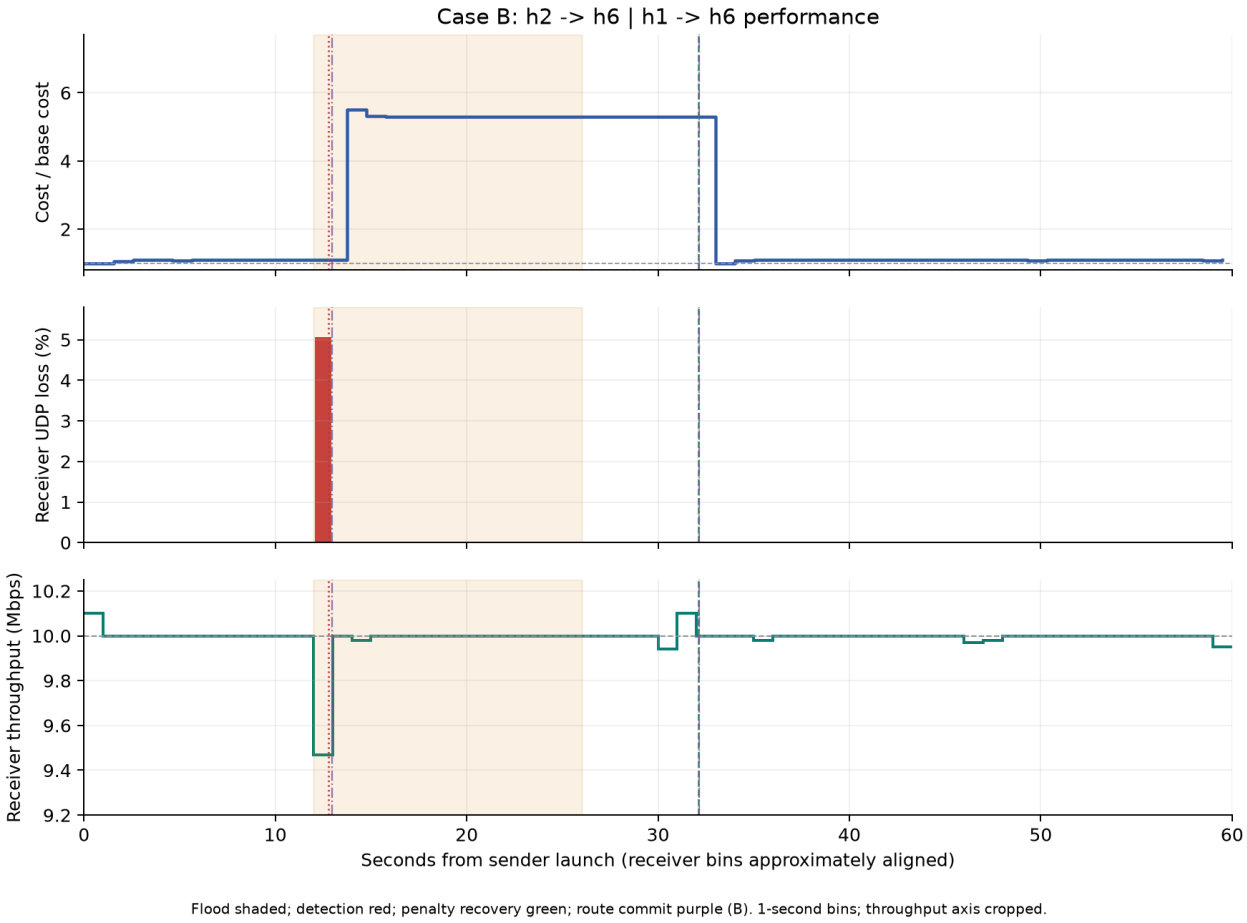


图 6 场景 B 的目标链路归一化代价、接收端 UDP 丢包率和吞吐。B 图使用第 1 轮。阴影表示计划 12–26 秒 flood 时段，事件标记来自控制器记录。

合法路线依次为 s1→s2→s6、s1→s2→s3→s6、s1→s2→s6。异常惩罚使原路变贵，替代路线避开 s2→s6；撤销惩罚后，流量返回重新变得便宜的原路线。

60 秒接收报告为 9.99 Mbps，UDP 丢失 43/51,024（0.0843%）。恢复核验时源入口 drop 计数为 888,103。测量后合法 ping 收到 3/3，异常源收到 0/2；所有采样交换机的源封锁规则均保留。B 的最低一秒吞吐为 9.47 Mbps，最高一秒丢包率为 5.0708%。

4 场景 C 异常位于 s5 至 s6

由 h5→h6 产生 flood，目标链路为 s5→s6。检测触发源封锁，并对目标边施加异常惩罚。目标边空闲、惩罚生效和恢复后的代价依次为 20 → 约 80 → 20。正常合法流量经过的链路还包含实测负载项。

目标链路采样到达速率峰值为 9.17 Mbps；异常源检测速率为 25,340.8 pps，阈值为 2000 pps。入口封锁且在途流量排空后，目标链路到达速率降至接近空闲，而惩罚继续保持至满足安静期。

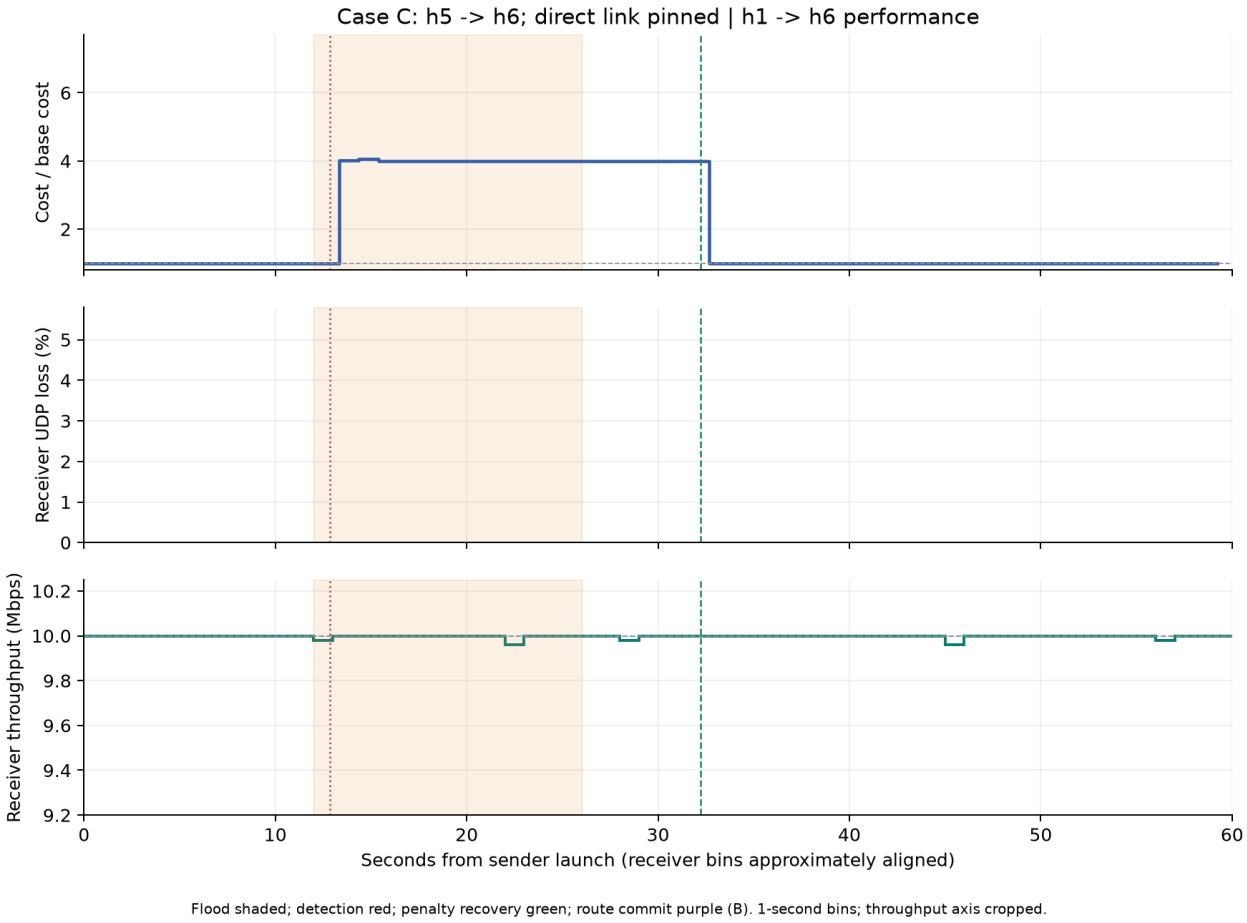


图 7 场景 C 的目标链路归一化代价、接收端 UDP 丢包率和吞吐。B 图使用第 1 轮。阴影表示计划 12–26 秒 flood 时段，事件标记来自控制器记录。

合法路线保持 s1→s2→s6，因为该路线不使用 s5→s6。异常源为 h5，其源封锁不会匹配 h1 流量。本场景是在前述测试对固定路径条件下，对作业指定链路施压。

60 秒接收报告为 10 Mbps，UDP 丢失 0/51,024（0.0000%）。恢复核验时源入口 drop 计数为 898,283。测量后合法 ping 收到 3/3，异常源收到 0/2；所有采样交换机的源封锁规则均保留。

5 Task 5 性能指标与阶段比较

链路归一化代价为 $C_{ij}(t)/C^{0}_{ij}$，A、B、C 分母分别为 10、14、20。数值 1 表示基础路由代价，并非 100% 利用率。相同绝对异常惩罚会在不同场景形成不同归一化高度，不能据此比较攻击强弱。

$$\text{UDP loss (\%)} = 100 \times \text{lost datagrams} / \text{total datagrams}$$

吞吐取接收端 iperf 的 Mbps 报告，不以发送端配置速率代替。曲线保留原始一秒窗口，不做平滑。整轮丢包使用客户端转述的最终服务端报告；接收端逐秒窗口的末端边界可能略有不同。吞吐保留原始显示精度，下表稳定窗口吞吐为接收窗口报告按持续时间加权平均。

实验	整轮 Mbps	UDP 丢失 / 总数	丢包率 %
A	10.0	0 / 51,024	0.0000
B1	9.99	43 / 51,024	0.0843
B2	9.99	78 / 51,024	0.1529
B3	10.0	22 / 51,024	0.0431
C	10.0	0 / 51,024	0.0000

实验	异常前 4–10 秒	异常中 15–24 秒	异常后 40–55 秒
A	10.000	10.000	9.997
B1	10.000	10.000	9.997
B2	10.000	10.000	10.000
B3	10.000	10.000	10.000
C	10.000	9.996	9.997

第二张表的数值单位为 Mbps，所列每个稳定窗口的 UDP 观测丢包率均为 0%。这些窗口有意避开约 12–14 秒启动过渡段；稳定窗口零丢包不能否认完整时间曲线中的短时丢包。

A、C、B2、B3 以发送进程启动时间为时间原点；B1 因原始启动元数据缺失，按采样元数据估计。接收窗口从第一份数据报到达开始计时。因此控制器事件与接收窗口仅约按一秒级对齐，没有毫秒级同步。阴影终点是计划 timeout 时间，不是独立测得的进程退出时刻。

5 路径代价解释 B 场景的改道

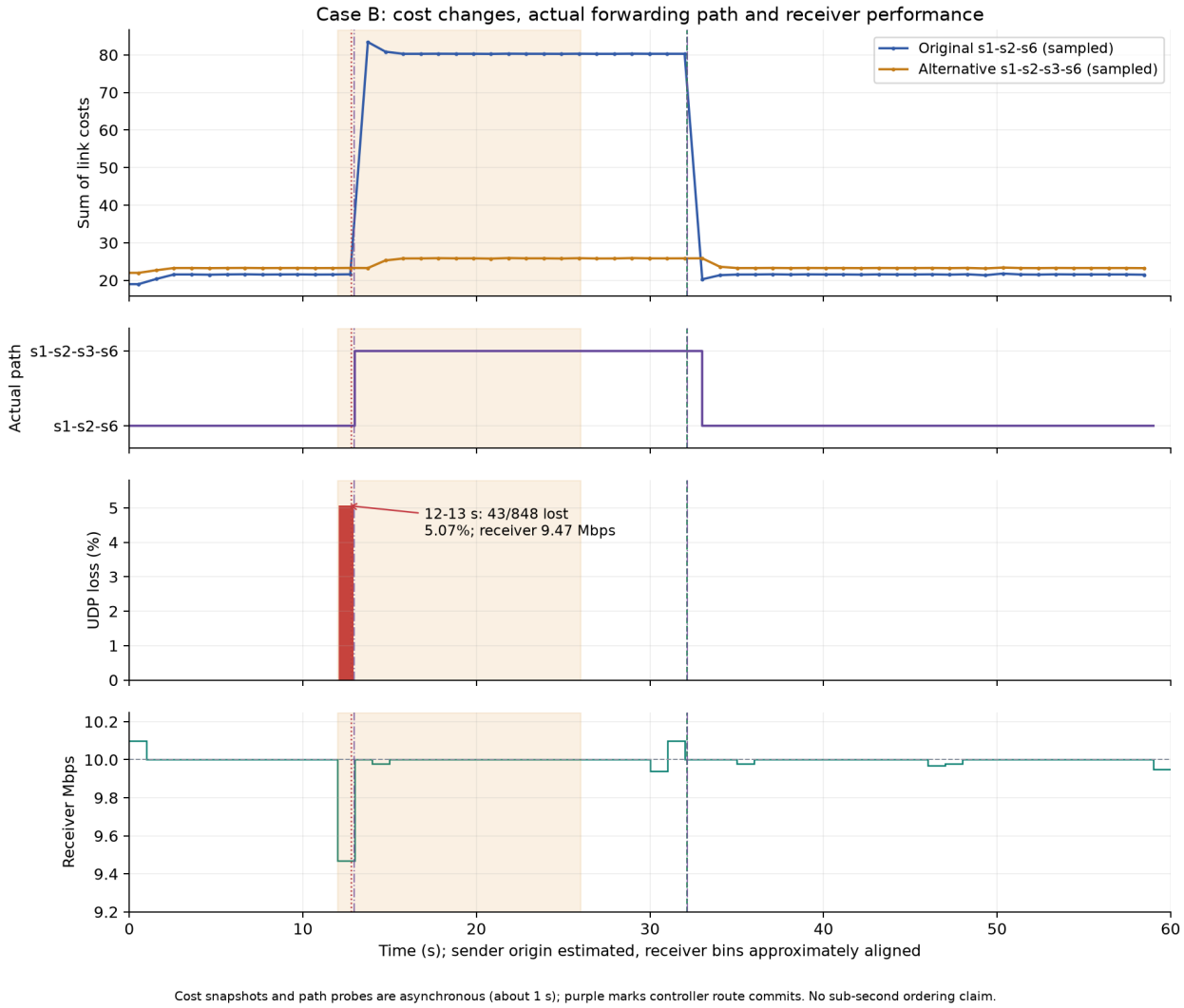


图 8 B 场景第 1 轮：候选路线代价、实测安装路线与接收性能。

正常稳定阶段，原路线动态代价约 21.57，替代路线约 23.28。异常惩罚生效时，原路线约 80.35，替代路线约 25.80，因此切换符合当前更低总代价的选路原则。恢复后约 21.58 对 23.29，原路线重新占优。这些数值包含负载项，基础代价仍分别为 19 和 22。

路径条带由实际入口标签、下游输出规则、接口连接关系和递增包计数共同支持，并非仅从控制器 ROUTE 打印推断。代价快照与流表探测是异步的，因此不能利用图中细微错位声称亚秒级因果先后。

5 B 场景的重复测量

三轮独立 B 实验使用相同名义配置，图中保留全部结果，包括最初一轮和表现较差的一轮。重复测量用于补充作业要求的三种异常位置，不能替代 A、C 场景。

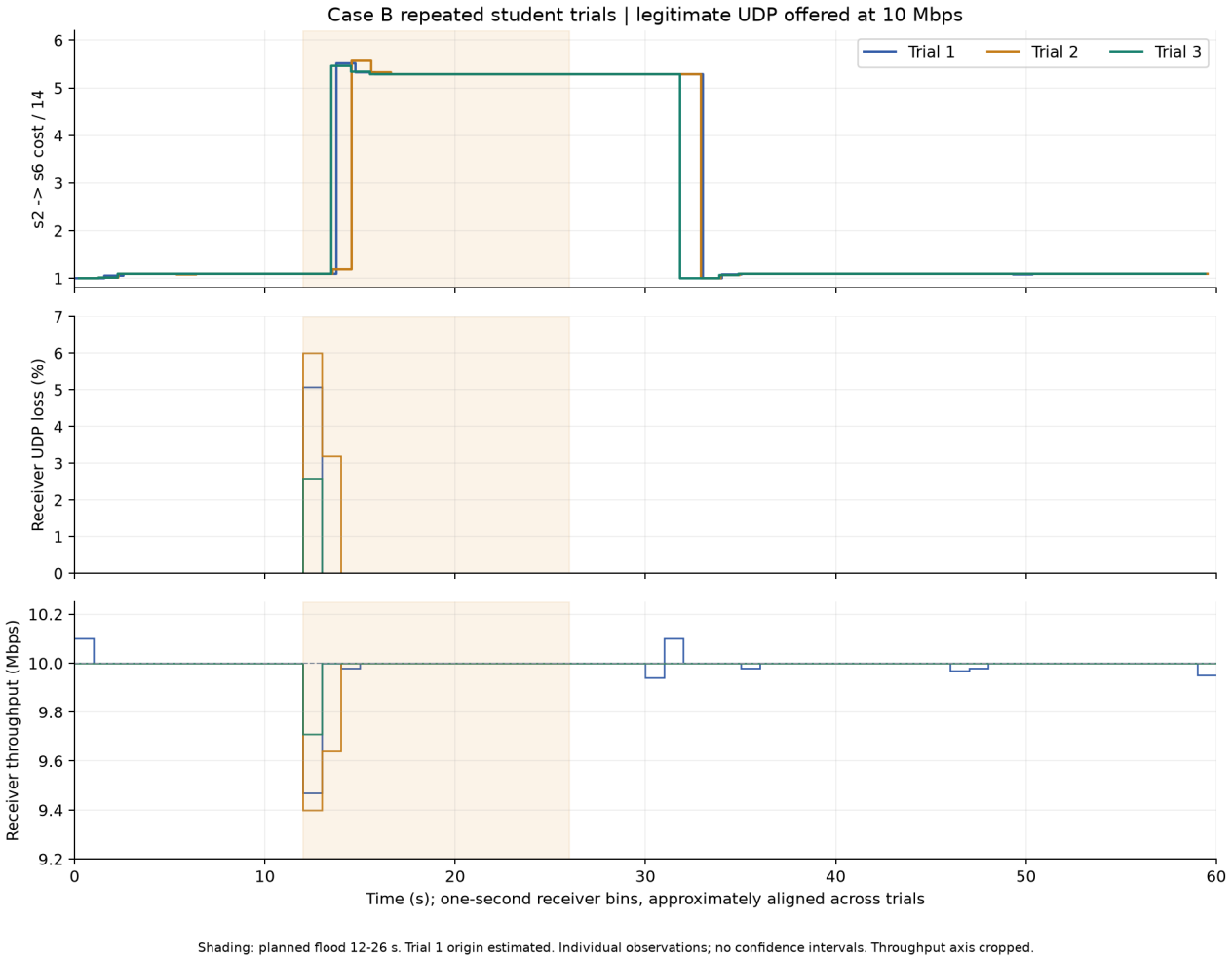


图 9 三轮 B 实验的 s2→s6 归一化代价、UDP 接收丢包率和吞吐。曲线分别代表各轮观测，不是合并平均曲线。

轮次	最高一秒丢包率 %	最低一秒 Mbps	源入口丢弃包数
B1	5.0708	9.47	888,103
B2	6.0000	9.40	825,016
B3	2.5943	9.71	901,466

三轮均切换至 s1→s2→s3→s6，随后返回 s1→s2→s6。整轮丢包率均值为 0.0934%，样本标准差为 0.0554 个百分点，范围 0.0431%–0.1529%。合并丢失 143/153,072 个数据报，比例同为 0.0934%。这些只是三次观测的描述统计，不提供置信区间或长期可靠性保证。

5 丢包 连通性与恢复的解释

短时过渡与稳定接收

B1 在 12–13 秒丢失 43/848 个数据报；B2 在 12–13 秒丢失 51/850、13–14 秒丢失 27/847；B3 在 12–13 秒丢失 22/848。三轮 B 的接收时间曲线中，其余窗口未观察到 UDP 丢包。全部一秒窗口吞吐大于零，异常稳定阶段恢复至约 10 Mbps。

flood 命令没有固定每秒发送包数，CPU 调度、流量生成、队列状态及轮询相位均可能随实验变化。证据表明扰动位于异常启动附近，但无法为每个丢包确定单一原因。没有进行“保留相同检测封锁而关闭改道”的对照实验，因此无法量化改道相对于仅封锁的独立收益。

合法连通与源隔离分别验证

每轮集成实验测量后，h1→h6 ping 均为 3/3，异常源均为 0/2。采样显示改道、旧版本清理和恢复过程中，各交换机源封锁项均保留。合法持续接收与异常源探测受阻共同支持选择性隔离，但不表示检测决策前没有任何攻击包通过。

每轮集成实验的持续 ICMP 原始日志均为发送 121、接收 120。最后一个序号未收到，与清理时仍有在途包的情况相容，但没有抓包确定其原因。ICMP 统计与 UDP 丢包曲线分别记录，因此报告不声称 ICMP 零丢包。

恢复代价但保留封锁

flood 停止且满足安静期后，异常代价项撤销，合法路线可以恢复；异常源保持封锁，直到新一轮重启状态。已封锁源的新包会对最后记录的攻击路径重新激活短期惩罚。因此脚本在 60 秒后发送的两次异常源 ping 可能引起额外一次路线切换；这些事件不计入正式 0–60 秒性能窗口。

5 A 与 C 的学生实验汇总截图

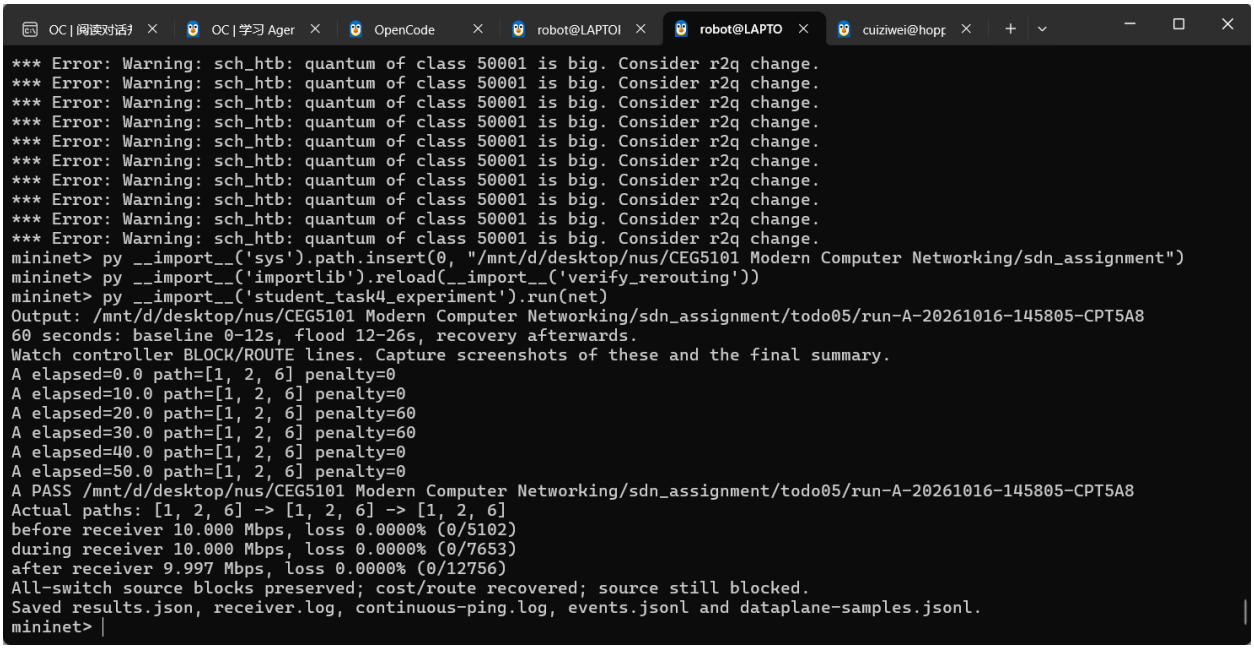


图 10 A 场景学生终端最终汇总：合法路线保持不变，并列出异常前中后接收指标。

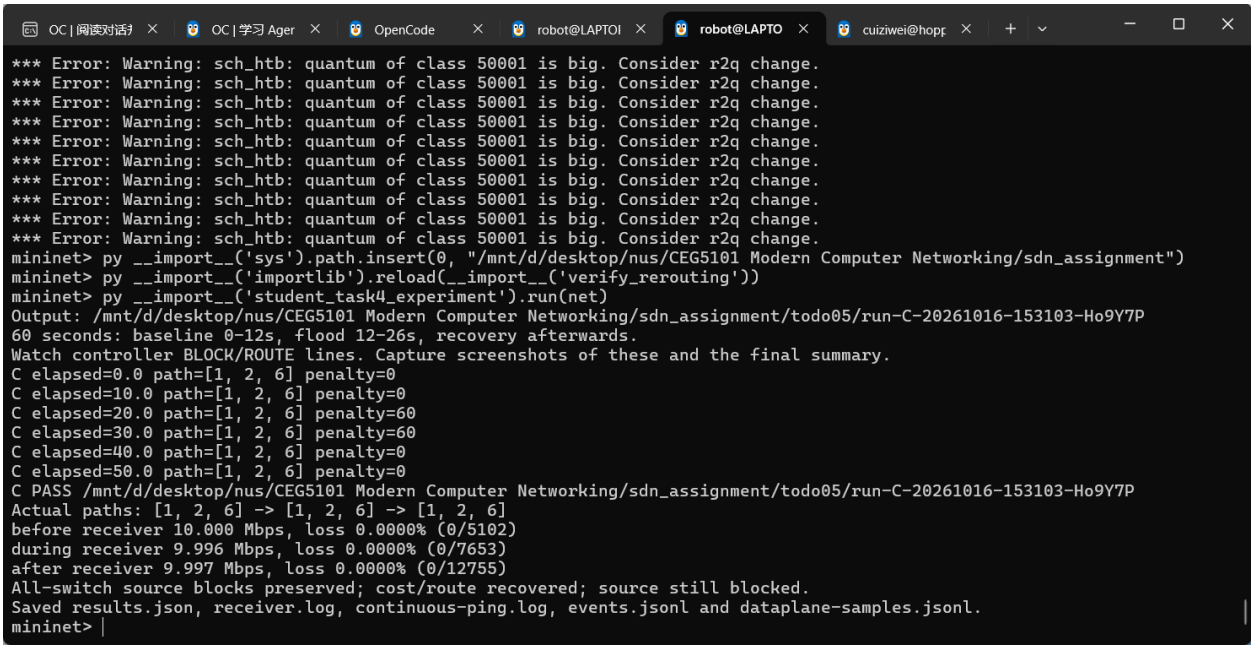


图 11 C 场景学生终端最终汇总，实验采用已披露的直接链路测试条件。

PASS 输出只是辅助核验，不能代替分析。两种场景的受影响边都位于 h1→h6 原路之外，因此保持原路线符合动态最短路原则。前文代价和接收曲线提供了逐时证据。

5 B 场景与源封锁证据

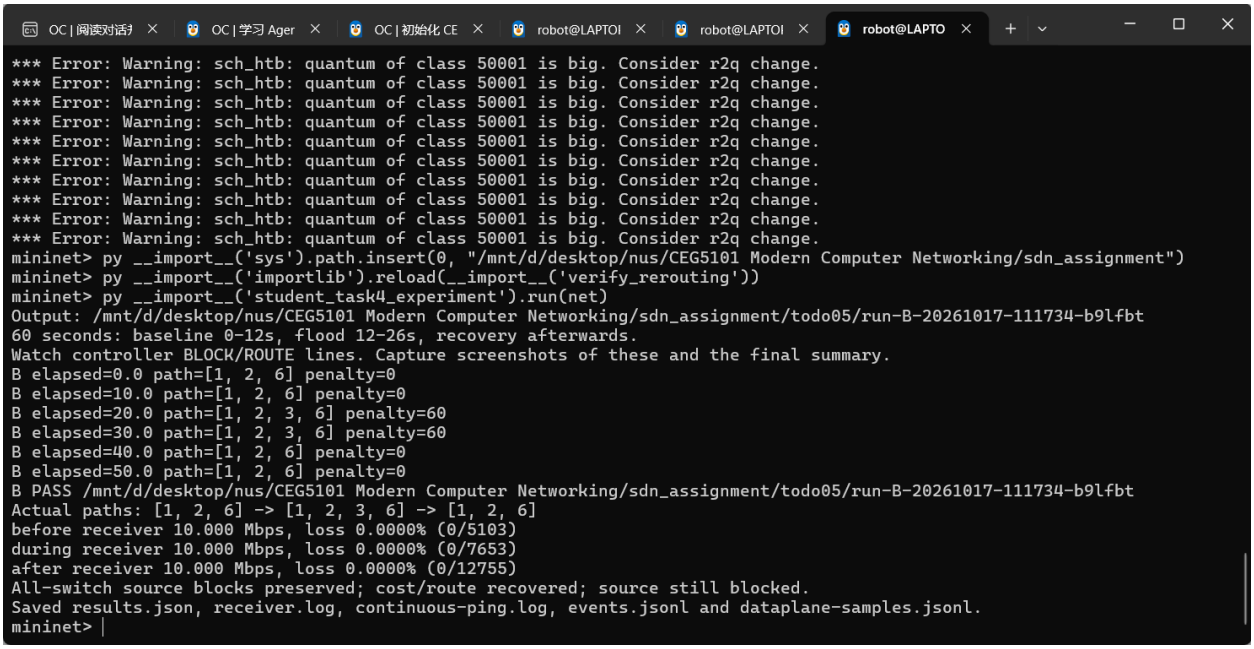


图 12 B 场景第 3 轮终端汇总，确认改道、恢复及持续接收。

Task 2 终端证据摘录

s2：8 秒 h2 flood 后检查流表
cookie=0xca230002 priority=5000
ip,nw_src=10.0.0.2 actions=drop
n_packets=478615 n_bytes=20101830
h2 → h6 ping：发送 3，接收 0
h1 → h6 ping：发送 3，接收 3

上述为学生粘贴终端记录的排版摘录，并非重新制作的截图 [6]。drop 项没有转发动作。较低优先级的 h2→h6 转发项仍可存在，但高优先级源规则先匹配。规则存在、drop 计数递增和异常源 ping 失败共同证明封锁生效。

B1 最初打印 FAIL，原因是检查器误要求本地 ryu.log，而控制器实际运行在学生的外部终端。原 FAIL 记录保留，另存离线核验结果，检查已有事件、流表快照、接收数据与探测；之后 B2、B3 直接打印 PASS。后续成功实验没有覆盖早期观测。

6 结论 适用范围与 AI 使用声明

测试结果支持控制器将源流量监测、EMA 异常检测、IPv4 源封锁、负载感知代价和条件式合法改道集成实现。A、C 隔离异常，同时保持未受影响的合法路线；B 的三轮测量均显示异常期间选择更便宜的替代路线，并在之后恢复原路。合法 UDP 稳定接收约为 10 Mbps，B 在启动附近有短暂丢包。

评价仅覆盖一个拓扑和 10 Mbps 合法提供负载，没有进行 40 Mbps 饱和基准。A、C 各一轮，只有 B 重复三轮。参数有运行层面的理由，但没有通过扫描证明最优。未评价物理链路故障：当前固定拓扑实现遇到就绪后交换机或链路消失时会停止路由事务。C 的受约束测试对仍需教学确认；封锁后惩罚重激活还假设已记录攻击路径继续适用。

AI 辅助使用声明

使用 OpenAI ChatGPT 和 Codex 辅助解释 SDN 概念、配置与排查环境、编写和修改控制器及实验脚本、进行自动核验、处理已保存实验数据、生成图表，以及起草和排版本报告。报告中的测量与截图来自学生操作实验，助手独立验证不计入学生重复实验。提交者负责理解、核对并承担最终提交内容及质量的责任。该声明依据已提供的学术规范说明 [8]。

参考资料与原始证据

[1] CEG5101，CA2 Laboratory Assignment，Anomaly aware Routing and Blocking Controller，2026/27 第一学期，第 1–4 页。
[2] 教师实验资料：ceg_mesh_topo.py。
[3] 教师实验资料：anomaly_detection.py。
[4] 教师实验资料：resilient_controller.py。
[5] 学生正常基线证据，todo01、todo02，2026 年 10 月 11、16 日。
[6] 学生异常检测证据，todo03，2026 年 10 月 16 日。
[7] 学生动态代价证据，todo04，2026 年 10 月 16 日。
[8] 已提供的学术规范说明：学术规范.txt。
[9] 学生集成实验 todo05 与最终 Task 5 分析 todo06/task5-final-20261017，2026 年 10 月 16–17 日。

附录 复现步骤与证据索引

环境为 WSL2 下的 Ubuntu 22.04、Mininet 2.3.0、Open vSwitch 2.17.12。MiniEdit 使用系统 Python 3.10.12；控制器使用独立环境 Python 3.8.20、Ryu 4.34、eventlet 0.30.2 和 NetworkX 2.6.3。教师源码原件保留不变；各阶段分别由 baseline_controller.py、anomaly_controller.py、dynamic_cost_controller.py 和 rerouting_controller.py 实现。

```
Ubuntu terminal A: sudo miniedit
Open /home/robot/CEG5101/todo01/ca2-mesh-my.mn
Ubuntu terminal B, from the assignment workspace:
bash run_rerouting.sh A # then repeat with B and C
In MiniEdit: Run, wait for REROUTING READY and warm-up
In mininet> (insert the workspace into sys.path first):
py __import__('student_task4_experiment').run(net)
```

Python 模块调用会在当前 MiniEdit 的网络对象 net 上运行定时流量和观测流程，它不是 Ryu 控制器入口。场景之间先退出 Mininet CLI、点击 Stop、停止上一 Ryu 进程，再启动下一场景。正式实验记录持续 60 秒的合法流、约第 12 秒开始的 14 秒 flood、逐秒流表与端口快照，以及接收端报告。

实验	todo05 下的原始实验目录
A	run-A-20261016-145805-CPT5A8
B1	run-B-20261016-144402-YuFNPf
B2	run-B-20261017-105957-DfB6RZ
B3	run-B-20261017-111734-b9lfbt
C	run-C-20261016-153103-Ho9Y7P

各集成实验保存 sender.log、receiver.log、continuous-ping.log、dataplane-samples.jsonl、初始与恢复证据，以及另存的核验结果。事件快照和 B 比较输入被冻结，避免控制器后续追加日志改变分析来源。插图按原文件字节复制至报告 assets 目录，并记录 SHA256。最终 Task 5 目录还保存逐秒 CSV、原始 SVG、汇总 JSON 和核验记录。

要求覆盖：拓扑与配置见第 1 节；源速率、EMA 和封锁见第 2 节；负载与异常代价见第 3 节；三个指定异常链路、选路与恢复见第 4 节；归一化代价、丢包、吞吐曲线与前中后解释见第 5 节。整理报告期间未新增流量实验。